Prefetching in 3-D Spatial Algorithms

Tahereh Vasei

Abstract

This report details the design and implementation of a prefetcher for 3-D spatial algorithms using SystemVerilog. The prefetcher predicts and fetches data from slow memory to fast memory, improving access efficiency for spatial data structures.

1 Introduction

Prefetching is a technique that anticipates the data needs of a processor and retrieves it from slow memory to fast memory. This report presents a prefetcher designed for 3-D spatial algorithms, highlighting the implementation in SystemVerilog and the testing process.

2 Design and Implementation

2.1 Prefetcher Module

The 3D spatial prefetcher is designed to predict adjacent addresses in a 3D space. The following sections describe the modules and their functionality.

The implementation consists of two main components: the datapath and the controller. The datapath calculates adjacent addresses, while the controller manages state transitions.

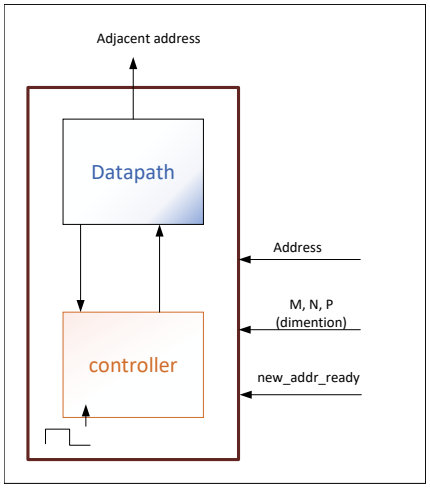

Figure 1: Top Module Diagram

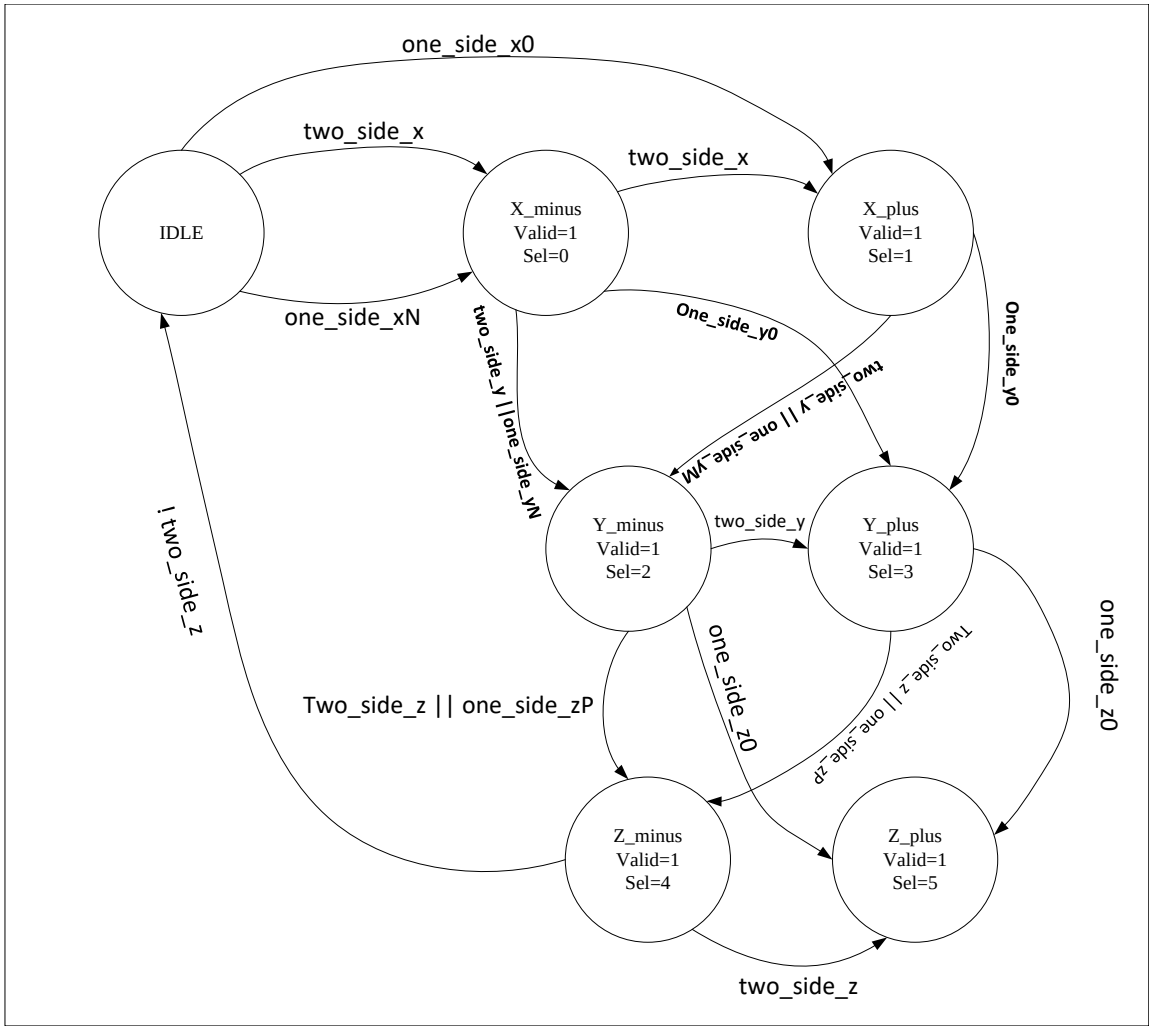

Figure 2: Controller Diagram

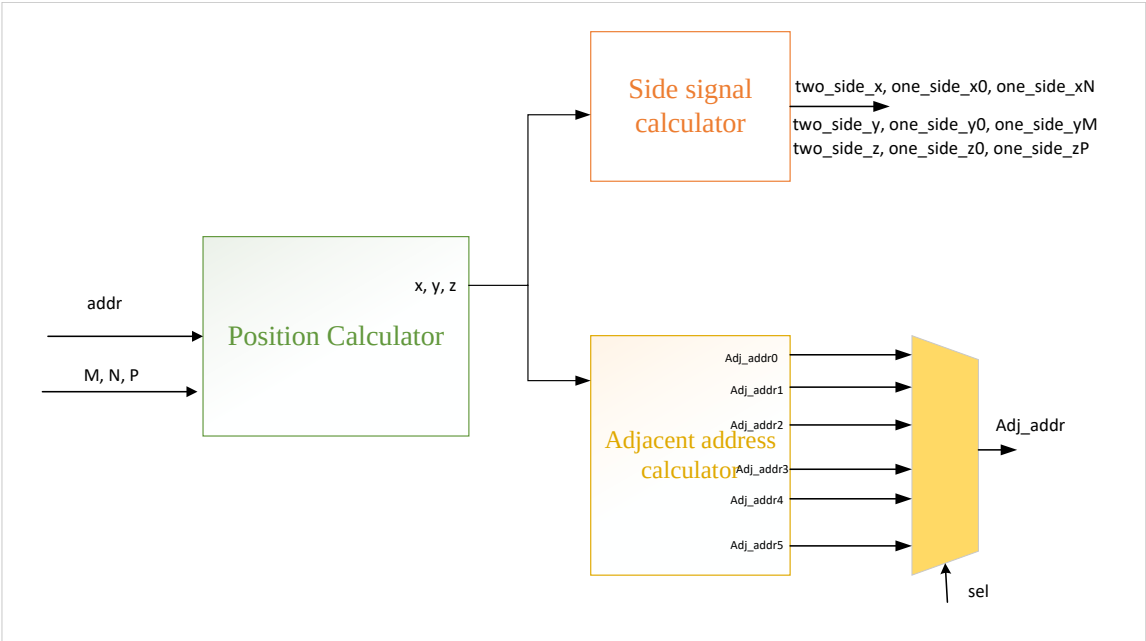

Figure 3: Datapath Diagram

The pseudocode outlines a 3D spatial prefetcher algorithm. It calculates positions and determines adjacent addresses based on current coordinates. The state machine manages transitions to fetch each adjacent address efficiently, ensuring data is ready in cache when needed.

```
Initialize variables and states

Main Loop:
  While running:
    If new address is ready:
      Calculate x, y, z from address
      Determine side signals:
        two_side_x, one_side_x0, one_side_xN
        two_side_y, one_side_y0, one_side_yM
        two_side_z, one_side_z0, one_side_zP

      Calculate adjacent addresses:
        adj_addr_array[0] = (x > 0) ? addr - 1 : addr
        adj_addr_array[1] = (x < M-1) ? addr + 1 : addr
        adj_addr_array[2] = (y > 0) ? addr - M : addr
        adj_addr_array[3] = (y < N-1) ? addr + M : addr
        adj_addr_array[4] = (z > 0) ? addr - M * N : addr
        adj_addr_array[5] = (z < P-1) ? addr + M * N : addr

    State Machine:
      Case current_state:
        IDLE:
          If new address ready:
            If two_side_x or one_side_xN:
              next_state = X_minus
            Else if one_side_x0:
              next_state = X_plus

        X_minus:
          Set sel = 0
          Set valid = 1
          If two_side_x:
            next_state = X_plus
          Else if two_side_y or one_side_yM:
            next_state = Y_minus
          Else if one_side_y0:
            next_state = Y_plus

        X_plus:
          Set sel = 1
          Set valid = 1
          If two_side_y or one_side_yM:
            next_state = Y_minus
          Else if one_side_y0:
            next_state = Y_plus

        Y_minus:
```

```
        Set sel = 2
        Set valid = 1
        If two_side_y:
          next_state = Y_plus
        Else if two_side_z or one_side_zP:
          next_state = Z_minus
        Else if one_side_z0:
          next_state = Z_plus

      Y_plus:
        Set sel = 3
        Set valid = 1
        If two_side_z or one_side_zP:
          next_state = Z_minus
        Else if one_side_z0:
          next_state = Z_plus

      Z_minus:
        Set sel = 4
        Set valid = 1
        If two_side_z:
          next_state = Z_plus
        Else:
          next_state = IDLE

      Z_plus:
        Set sel = 5
        Set valid = 1
        next_state = IDLE
```

3 Testing and Verification

A test bench was created to verify the functionality of the prefetcher. Basic tests confirmed the correct output of adjacent addresses.

The following images depict the waveforms for different test cases:

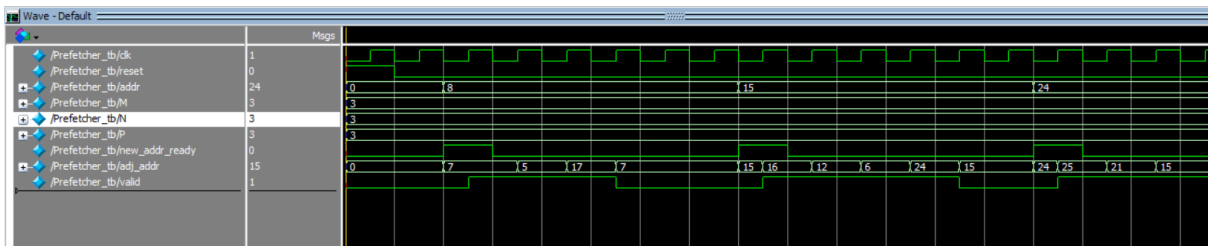

Figure 4: Waveform for 3*3*3 : Address 8, 15, 24

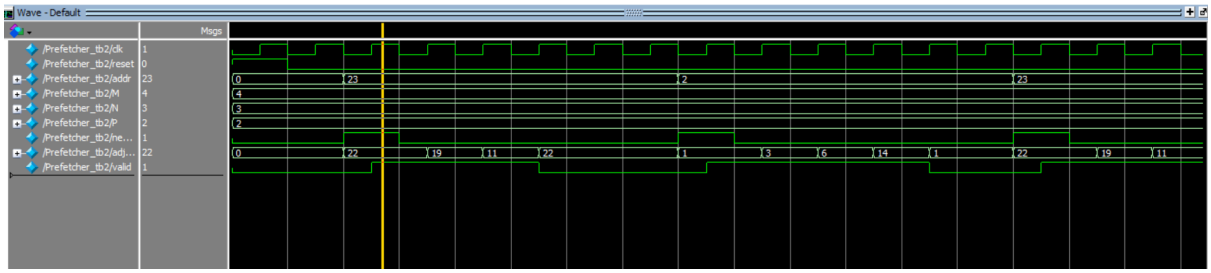

Figure 5: Waveform for 4*3*2: Address 23, 2

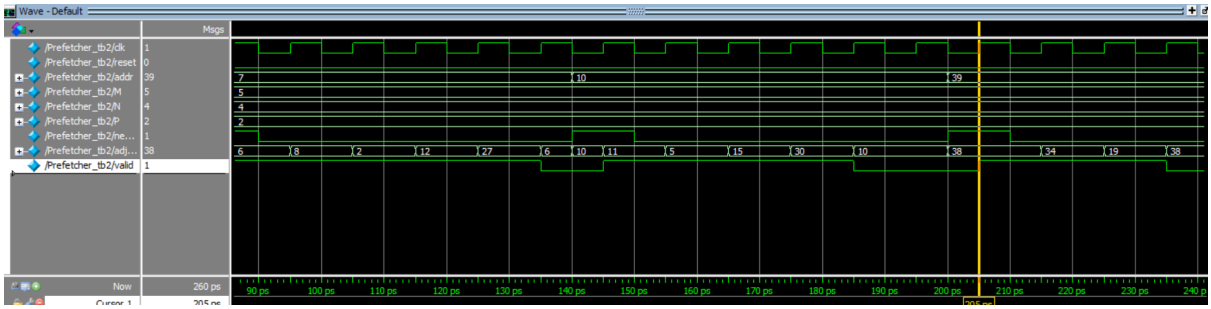

Figure 6: Waveform for 5*4*2: Address 7, 10, 39